Esercitazione Python n. 10 – Assegnata come lavoro autonomo

Per risolvere gli esercizi **scrivete la soluzione di ogni esercizio nel file .py relativo**, che trovate nella cartella dell'esercitazione (ad esempio, per l'esercizio 1 scrivete il vostro programma nel file Ex1.py, per l'esercizio 2, nel file Ex2.py, e così via). Notate che ogni file incorpora del codice python per eseguire alcuni test sulla funzione. NON modificate questo codice, ma **SCRIVETE SOLO il contenuto della funzione**. Per farlo usare l'ambiente Spyder.

Potete riconsegnare l'esercitazione attraverso il sito classroom. L'esercitazione non verrà comunque corretta singolarmente.

In ogni esercizio, se non diversamente richiesto, potete sempre assumere che gli input forniti siano coerenti con la traccia (ad esempio, se l'esercizio chiede di dare in input alla funzione un intero positivo, potete assumere che l'input sia sempre un numero intero maggiore di zero, e non è necessario nel codice effettuare controlli per gestire casi diversi da questo).

Esercizi

- **Ex1(fileName):** Scrivere una funzione che prende ingresso una stringa **fileName,** nome di un file csv, contenente le informazioni sugli esami superati dagli studenti (nel formato `Matricola,Voto,Materia`), e restituisce una lista contenente tutte e sole le coppie (`Matricola`, `Materia`) per gli esami superati, tali cioè che lo studente con quella matricola abbia preso un voto maggiore o uguale a 18. Notare che una coppia è semplicemente una tupla di lunghezza due. Le coppie devono comparire nella lista nello stesso ordine in cui le informazioni si trovano nel file. Ad esempio, se il file contiene

  ```
  Matricola,Voto,Materia
  1345,29,Fisica
  1987,17,Fondamenti
  1346,27,Analisi
  1896,30,Geometria
  1753,30,Fisica
  ```

  La funzione deve restituire

  ```
  [('1345','Fisica'),('1346','Analisi'),('1896','Geometria'),('1753','Fisica')]
  ```

- **Ex2(fileName):** Scrivere una funzione che prende in ingresso una stringa **fileName,** nome di un file csv che ha lo stesso formato del file descritto nell'esercizio precedente (**Ex1**), e restituisce un insieme contenente tutte e sole le `Materie` per cui ci sono almeno due studenti che hanno preso almeno 29 (potete assumere che nel file non ci siano mai due righe aventi stessa matricola e stessa materia – in altri termini, il file contiene un solo voto per un certo studente ed una certa materia).

- **Ex3(file, nome):** Una squadra di basket memorizza in un file di testo il numero di punti fatti da ogni giocatore in ogni partita. La struttura delle righe del file è la seguente:

  ```
  Nome1 punti1;Nome2 punti2;Nome3 punti3;.....;Nomen puntin
  ```

  Ogni riga del file rappresenta una partita e contiene i nomi di tutti i giocatori che hanno realizzato almeno un punto nella partita e, per ciascun nome, il numero di punti realizzati (notate che il nome ed i punti sono sempre separati da uno spazio). Scrivere una funzione che, presi in input il nome (parametro **file)** di un file contenente i punteggi delle partite secondo il formato appena descritto ed il nome (parametro **nome**) di un giocatore, restituisca il numero complessivo di punti realizzati da quel giocatore in tutte le partite (o 0 se il nome del giocatore non compare nel file). Ad esempio, se il contenuto di **file** è il seguente:

```
Anthony 30;James 23;Stephen 23;Jimmy 18
Luka 10;Bradley 21;James 23
Kareem 18;Magic 24;Shaquille 20;Michael 31
Anthony 13;Jimmy 20;Luka 33;James 10
Luka 10;Bradley 20;James 31
```

ed **n**=`'Luka'`, allora la funzione deve restituire `53`.

- **Ex4(file):** scrivere una funzione Python che prende in ingresso il nome di un **file** csv contenente le informazioni sulle amicizie e inimicizie che si creano in un gruppo di persone nel seguente formato:

```
Nome1,Nome2,relazione
```

dove la relazione può essere solo un valore tra `'amici'` e `'nemici'`. La relazione è sempre simmetrica ed una relazione riportata ad una certa riga può essere modificata da una relazione indicata in una riga successiva. Ad esempio, se **file** contiene:

```
Nome1,Nome2,relazione
Paolo,Marco,amici
Anna,Maria,amici
Paola,Anna,amici
Marco,Giorgio,amici
Giorgio,Marco,nemici
```

la riga 2 dice che Paolo è amico di Marco, e vice-versa (analogamente le altre righe), mentre notiamo che alla riga 5 Marco e Giorgio sono amici, ma alla riga 6 diventano nemici.
La funzione deve leggere il **file** e costruire un dizionario avente come chiavi i nomi di ***tutte*** le persone che compaiono nel file, e per valore associato a ciascuna chiave *k* la lista ordinata in ordine lessicografico crescente degli amici di *k* (rimasti al termine della lista). Ogni volta in cui 2 persone diventano amiche dovete aggiungere il nome di ciascuno dei due alla lista degli amici dell'altro, se non era già presente (non ci devono essere duplicati nella lista). Se due persone diventano nemiche dovete eliminare il nome di ciascuno dei due dalla lista degli amici dell'altro (se c'era). In riferimento al file d'esempio precedente, la funzione deve restituire il seguente dizionario:

```
{'Marco': ['Paolo'], 'Maria': ['Anna'], 'Paolo': ['Marco'], 'Paola':
['Anna'], 'Giorgio': [], 'Anna': ['Maria', 'Paola']}
```

Si ribadisce che nel dizionario restituito ci deve essere una chiave per ogni nome che compare nel file. Questo vale anche se una persona compare solo in righe in cui la relazione è `'nemici'`. In questo caso, ovviamente, la lista dei suoi amici sarà vuota.

- **Ex5(file)**: un indirizzo IP (semplificato) è un'etichetta numerica che identifica univocamente un dispositivo all'interno di una rete informatica. Esso è costituito da una sequenza di 4 numeri compresi tra 0 e 255 e separati da un punto (es. 192.168.0.1). Tra tutti i possibili indirizzi IP, vi è un intervallo dedicato alle reti domestiche, per le quali gli indirizzi IP hanno il formato 192.168.X.Y, dove X ed Y sono due numeri compresi tra 0 e 255. Scrivere la funzione Python che prende in ingresso il nome di un **file** di testo avente il seguente formato

```
Indirizzo_IP
Indirizzo_IP
    […]
Indirizzo_IP
```

E restituisce un dizionario contenente le seguenti chiavi: `'invalidi'`, `'domestici'` ed `'altri'`, e come valore associato a ciascuna chiave il numero di indirizzi letti dal file che sono rispettivamente non validi, domestici oppure validi non domestici. Se il file in input è vuoto, la funzione deve ovviamente restituire un dizionario in cui i valori sono tutti pari a `0`.

Potete assumere che ogni riga del file contenga solo cifre e punti (`.`). Un riga potrebbe quindi riportare un indirizzo IP non valido perché contiene un numero di sequenze di cifre diverso da 4, oppure perché la sequenza di cifre rappresenta un numero non compreso fra 0 e 255. Ad esempio, se il file contiene le seguenti righe

```
192.168.001.002
192.255.001.001
  255.255.255
100.001.256.002
301.001.002.100
```

La funzione deve restituire `{'invalidi': 3, 'domestici': 1, 'altri': 1}`.

- **Ex6(file):** scrivere una funzione Python che prende in ingresso un file di testo contenente dei codici fiscali, scritti uno per riga e senza spazi tra i vari campi, e restituisce la lista (nell'ordine in cui sono nel file) delle date di nascita nel formato dd/mm/aaaa (2 cifre obbligatorie per giorno e mese, 4 per anno). Si assuma che se l'anno xx nel codice fiscale è minore o uguale a 20 allora l'anno corrisponde a 20xx, altrimenti corrisponde a 19xx. Si ricorda che il formato dei codici fiscali è:

  ABCXYZaaMggWnnnV

  Dove ABC e XYZ sono sequenze di lettere maiuscole prese rispettivamente dal cognome e dal nome, aa denota le ultime due cifre dell'anno, M è una lettera maiuscola che specifica il mese secondo la tabella riportata a lato, gg denota il giorno di nascita con la regola che se è maggiore di 40 allora il sesso è femminile e per calcolare la data corretta bisogna togliere 40. L'ultima parte indica il codice del comune (o stato estero) di nascita, composto da una lettera maiuscola e 3 cifre, mentre l'ultima lettera maiuscola è un carattere di controllo. Potete assumere che i codici fiscali nel file siano sempre scritti secondo il formato precedente, ma dovete considerare che ci possono essere errori sulla lettera del mese e sul giorno. In particolare, se il codice del mese è inesistente (cioè non è contenuto nella tabella precedente), allora la funzione deve inserire nella lista la stringa `'Mese errato'`. Se invece il giorno è scorretto, allora la funzione deve inserire nella lista la stringa `'Giorno errato'`. Se sono scorretti sia il mese che il giorno, vale la regola per il mese scorretto (cioè, bisogna inserire 'Mese errato' nella lista). Nella verifica dei giorni potete ignorare gli anni bisestili ed assumere che Febbraio abbia sempre 28 giorni. Ad esempio, se il file contiene

Lettera	Mese	Lettera	Mese	Lettera	Mese
A	gennaio	E	maggio	P	settembre
B	febbraio	H	giugno	R	ottobre
C	marzo	L	luglio	S	novembre
D	aprile	M	agosto	T	dicembre

```
VXRTRR71C12H501W
PSCTRS21S18R330P
CVVPSX11D55H911T
CVVPSX11O55H911T
CVVPSX11D79H911T
```

  la funzione deve restituire `['12/03/1971', '18/11/1921', '15/04/2011', 'Mese errato', 'Giorno errato']`.

  Suggerimento: Per la soluzione potete usare il dizionario che trovate già definito nel file python relativo a questo esercizio, utile per decodificare i mesi a partire dalle lettere nella tabella. Ad ogni modo, è possibile usare anche strutture dati e meccanismi diversi per risolvere l'esercizio. Quindi, se non è chiaro come procedere usando il dizionario già definito, potete ignorarlo e fornire una soluzione differente.

- **Ex7(file)**: Scrivere una funzione Python che prende in ingresso il nome di un **file** csv contenente tutte le eredità di una famiglia nel seguente formato:

  `Oggetto,Antenato,Erede`

Il significato di una riga è che un certo oggetto passa da un antenato ad un erede. L'ordine delle righe nel file è importante: il proprietario originale dell'oggetto è l'antenato che compare la prima volta (dall'inizio del file) assieme all'oggetto. Vale quindi la seguente regola: in ciascuna riga l'antenato A possiede un oggetto solo se A è il proprietario originale, oppure se esiste una riga precedente in cui A riceve l'oggetto da un antenato che ha l'oggetto. Notate quindi che nel file ci possono essere righe in cui l'antenato menzionato NON ha l'oggetto (perché non lo ha a sua volta ereditato da qualcuno che lo aveva, secondo la regola scritta in precedenza) allora l'erede NON lo riceve.

La funzione deve leggere il **file** e restituire il dizionario con chiavi i nomi degli oggetti nel **file** e come valore associato a ciascuna chiave *k* una lista contenente due nomi, il nome del proprietario e il nome dell'ultimo erede che ha ricevuto l'oggetto *k*. Ad esempio se il **file** contiene:

```
Oggetto,Antenato,Erede
Anello_di_smeraldi,Maria,Paola
Anello,Silvia,Paolo
Anello_di_smeraldi,Paola,Anna
Anello_di_smeraldi,Anna,Giorgia
```

la funzione deve restituire: `{'Anello_di_smeraldi': ['Maria','Giorgia'], 'Anello': ['Silvia', 'Paolo']}`

Invece se il **file** contiene:

```
Oggetto,Antenato,Erede
Anello_di_smeraldi,Maria,Paola
Anello,Silvia,Paolo
Anello_di_smeraldi,Anna,Giorgia
Anello_di_smeraldi,Paola,Anna
```

la funzione deve restituire:
`{'Anello_di_smeraldi':['Maria','Anna'],'Anello':['Silvia','Paolo']}`
(infatti, alla riga `Anello_di_smeraldi,Anna,Giorgia` Anna non ha l'oggetto, perché non lo ha ricevuto in una riga precedente, e quindi va ignorata).

- **Ex8(fileName)** Scrivere una funzione che riceve in input una stringa **fileName,** nome di un file di testo, contenente in ogni riga una serie di numeri interi separati da virgole, e restituisce un dizionario che ha come chiavi i numeri (in formato intero) che appaiono nel file e come valore associato a ciascuna chiave *k* un insieme contenente i numeri di riga in cui appare *k*. Assumete che il numero identificativo delle righe parta dal valore 1. Ad esempio se il file contiene

  ```
  10,-5,-5,0
  10,-5,8,-3
  ```

  la funzione deve restituire:

  ```
  {10: {1,2}, -5: {1,2}, 0: {1}, 8: {2}, -3: {2}}
  ```

- **Ex9(fileName)** Scrivere una funzione che prende in ingresso una stringa **fileName,** nome di un file csv, contenente tutte le partite giocate in un torneo di calcio. Il file ha il seguente formato:

  Prima riga:
  `Nome_Squadra1,Nome_Squadra2,Numero_gol_Squadra1,Numero_gol_Squadra2`
  Altre righe:
  `Squadra1,Squadra2,golSquadra1,golSquadra2`

  La funzione deve restituire il dizionario che ha come chiavi i nomi delle squadre che compaiono nel file csv e come valore associato a ciascuna chiave *k* i punti in classifica della squadra *k*. Assumete che la squadra che vince ottiene 3 punti, la perdente ottiene 0 punti, e, in caso di pareggio, entrambe ottengono 1 punto.
  Ad esempio se file contiene

```
Nome_Squadra1,Nome_Squadra2,Numero_gol_Squadra1,Numero_gol_Squadra2
Chelsea,Everton,2,0
Arsenal,Tottenham,0,0
Chelsea,Arsenal,0,1
Tottenham,Everton,1,2
```

la funzione deve restituire:

```
{'Chelsea': 3, 'Everton': 3, 'Arsenal': 4, 'Tottenham': 1}
```